

학습 목표

딥러닝의 구조와 기본 원리의 이해를 높이기 위해 딥러닝 프레임워크를 사용하지 않고 Numpy만을 사용하여 딥러닝(회귀 문제)를 예측하는 것을 실습하는 것을 목표로 한다.

선행 지식

- [손실 함수 \(Loss function\)](#)
- [최적화 알고리즘 \(Optimization Algorithm\)](#)
- [활성화 함수 \(Activation Function\)](#)
- [역전파 \(backpropagation\)](#)

1. 데이터 불러오기

```
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing()
X = housing.data
y = housing.target
```

- california_housing 데이터를 편하게 불러오기 위해 sklearn을 사용하여 불러왔다.

2. 데이터 확인

```
import pandas as pd

# housing 데이터셋에서 특성 이름 출력
print("feature names: ", housing.feature_names)

print("\n\n")
```

```

# X의 데이터 타입과 크기 확인
print("Data type of X:", type(X))
print("Shape of X:", X.shape)

# y의 데이터 타입과 크기 확인
print("Data type of y:", type(y))
print("Shape of y:", y.shape)

print("\n\n")

# X와 y의 결측값 개수 확인
print("Number of missing values in X:\n", pd.DataFrame(X,
columns=housing.feature_names).isnull().sum())
print("Number of missing values in y:",
pd.Series(y).isnull().sum())

```

- 특성 이름 확인
 1. **MedInc**: 블록 그룹 내 가구들의 중간 소득, 단위는 천 달러
 2. **HouseAge**: 블록 그룹 내 주택들의 중간 연령
 3. **AveRooms**: 블록 그룹 내 평균 방의 개수
 4. **AveBedrms**: 블록 그룹 내 평균 침실의 개수
 5. **Population**: 블록 그룹 내 인구수
 6. **AveOccup**: 블록 그룹 내 평균 주택점유율
 7. **Latitude**: 블록 그룹의 위도
 8. **Longitude**: 블록 그룹의 경도

- 데이터 크기 확인

```

Data type of X: <class 'numpy.ndarray'>
Shape of X: (20640, 8)
Data type of y: <class 'numpy.ndarray'>
Shape of y: (20640,)

```

- 20640의 주택 데이터(8개의 클래스를 가진)가 존재
- 결측값 확인

```

Number of missing values in X:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
dtype: int64
Number of missing values in y: 0

```

- 결측값이 없으므로 결측값을 지우는 과정은 생략한다.

3. Data Scaling

```

from sklearn.datasets import fetch_california_housing
import numpy as np

# California Housing 데이터셋 로드
housing = fetch_california_housing()
X, y = housing.data, housing.target

# Min-Max Scaling 수행
X_min, X_max = X.min(axis=0), X.max(axis=0)
X_scaled = (X - X_min) / (X_max - X_min)

y_min, y_max = y.min(), y.max()
y_scaled = (y - y_min) / (y_max - y_min)

print("Scaled X:", X_scaled)
print("Scaled y:", y_scaled)

```

- NumPy를 사용하여 MinMaxScale을 구현하여 각각을 0과 1사이의 값으로 Scaling해준다.

- $y(\text{target})$ 값도 같이 Scale해주면 예측한 이후에 다시 역변환 해주어야 한다.

5. 이상치 확인

```
from scipy.stats import zscore

# X와 y의 z-score 계산
z_scores_X = zscore(X, axis=0)
z_scores_y = zscore(y, axis=0)

# z-score의 절대값이 2보다 큰 값을 찾기
outliers_X = (np.abs(z_scores_X) > 2).any(axis=1)
outliers_y = np.abs(z_scores_y) > 2

# X와 y에서 동일한 위치의 이상치를 찾기
outliers = outliers_X | outliers_y

# 이상치를 제거
X_clean = X[~outliers]
y_clean = y[~outliers]

print("Original size:", X.shape[0])
print("Size after removing outliers:", X_clean.shape[0])
```

6. 데이터 분할

```
import numpy as np

X = np.array(X_clean)
y = np.array(y_clean).reshape(-1, 1)
```

```
# 데이터를 훈련 세트, 검증 세트, 테스트 세트로 분할
train_size = int(len(X) * 0.7)
val_size = int(len(X) * 0.15)
test_size = len(X) - train_size - val_size

X_train, X_temp = X[:train_size], X[train_size:]
y_train, y_temp = y[:train_size], y[train_size:]
X_val, X_test = X_temp[:val_size], X_temp[val_size:]
y_val, y_test = y_temp[:val_size], y_temp[val_size:]

print("훈련 세트 크기:", X_train.shape, y_train.shape)
print("검증 세트 크기:", X_val.shape, y_val.shape)
print("테스트 세트 크기:", X_test.shape, y_test.shape)
```

- 훈련시 early stopping을 사용할 것이기 때문에 validation set 또한 지정 해주었다.

```
훈련 세트 크기: (12201, 8) (12201, 1)
검증 세트 크기: (2614, 8) (2614, 1)
테스트 세트 크기: (2616, 8) (2616, 1)
```

7. 모델

7.1 layer size 지정 weight,bias 초기화

```
import numpy as np

class SimpleNN:
    def __init__(self, layer_sizes, learning_rate=0.01,
initialization='he'):
        # 매개변수 및 하이퍼파라미터 초기화
        self.params = {} # 가중치와 편향을 저장할 딕셔너리
        self.learning_rate = learning_rate # 경사하강법의 학습률
```

```

print("다음 구조를 가진 신경망을 구축합니다:")
# 입력층
print(f"입력층: {layer_sizes[0]} 뉴런")

# 각 층의 가중치와 편향 초기화
for i in range(1, len(layer_sizes)):
    # 가중치 초기화
    if initialization == 'he':
        self.params['W' + str(i)] =
np.random.randn(layer_sizes[i-1], layer_sizes[i]) * np.sqrt(2. /
layer_sizes[i-1])
    elif initialization == 'xavier':
        self.params['W' + str(i)] =
np.random.randn(layer_sizes[i-1], layer_sizes[i]) * np.sqrt(1. /
layer_sizes[i-1])
    else:
        raise ValueError("유효하지 않은 초기화 방법입니다.
'he' 또는 'xavier'를 선택하세요.")

    # 편향을 0으로 초기화
    self.params['b' + str(i)] = np.zeros((1,
layer_sizes[i]))

    # 층 정보 출력
    print(f"은닉층 {i} (해당되는 경우): {layer_sizes[i-1]}
뉴런 -> {layer_sizes[i]} 뉴런")

# 출력층
print(f"출력층: {layer_sizes[-1]} 뉴런")

```

- 지정한 layer_sizes 만큼의 weight와 bias의 초기화한다.
- init을 사용하여 호출시 처음에만 실행된다.
- 두 가지 초기화 방법으로 he와 xavier를 사용하는데 각각 ReLU 와 sigmoid를 사용할 때 더 좋은 모습을 보인다.

- bias는 초기와시에는 모든 값을 0으로 지정한다.
- layer_size를 [8, 64, 64, 32, 16, 8, 4, 1]로 지정했다면 각각의 가중치의 크기는 다음과 같다.
 - $W1 = 8 * 64$ 행렬
 - $W2 = 64 * 64$ 행렬
 - $W3 = 64 * 32$ 행렬
 - $W4 = 32 * 16$ 행렬
 - $W5 = 16 * 8$ 행렬
 - $W6 = 8 * 4$ 행렬
 - $W7 = 4 * 1$ 행렬
- bias의 크기는 다음과 같다.
 - $b1 = 1 * 64$ 행렬
 - $b2 = 1 * 64$ 행렬
 - $b3 = 1 * 32$ 행렬
 - $b4 = 1 * 16$ 행렬
 - $b5 = 1 * 8$ 행렬
 - $b6 = 1 * 4$ 행렬
 - $b7 = 1 * 1$ 행렬
- 이 weight와 bias행렬의 생성은 순전파(forward)시에 입력값과 같이 생각해보면 자연스럽다.

7.2 순전파 (forward pass)

```
def forward(self, X):
    # 신경망을 통한 순방향 전파
    activations = {'A0': X} # 활성화 값을 저장할 딕셔너리
    L = len(self.params) // 2 # 총 층의 개수
    for i in range(1, L + 1):
        # 선형 변환
        Z = np.dot(activations['A' + str(i-1)], self.params['W'
+ str(i)]) + self.params['b' + str(i)]
```

```

# 출력층이 아니면 ReLU 활성화 함수 적용
if i < L:
    A = self.ReLU(Z)
else:
    A = Z # 출력층에는 선형 활성화 함수 적용
activations['Z' + str(i)] = Z
activations['A' + str(i)] = A
return activations

```

- `forward` 함수는 초기 입력 데이터를 받아서 `__init__` 메서드에서 초기화한 가중치와 편향 행렬들을 차례로 통과시키며 계산을 수행하고, 최종적으로 출력층에서 결과값을 출력한다.
- 이 코드에서 설정된 배치 크기는 128이므로 주어진 문제에서 집값을 예측하기 위해 8개의 특성을 사용하기 때문에 입력 데이터의 크기는 128×8 행렬이 된다. 이 행렬은 첫 번째 은닉층의 가중치 행렬인 8×64 행렬과 곱해져서 128×64 행렬이 되고, 여기에 편향 행렬 1×64 이 더해진다.
- 활성화 함수를 통과한 결과는 다음 은닉층으로 전달되며, 이러한 과정이 모든 은닉층에서 반복된다.
- 마지막 출력층에서는 활성화 함수를 거치지 않고 바로 예측값이 출력된다.
- 코드 확인
 - 입력값 X 가 주어지면, 이는 먼저 초기화된 첫 번째 은닉층의 가중치와 곱해지고, 이후 편향이 더해진다.
 - 이런 과정이 마지막 층 L 에 도달할 때까지 반복되며, 각 층마다 활성화 함수를 통과한다.
 - 다만, 마지막 층에서는 활성화 함수를 거치지 않고 직접 결과값을 출력한다.
 - 각 층에서의 활성화 값을 모두 저장하는 이유는, 나중에 역전파 과정에서 이 값을 이용하여 미분 값을 효율적으로 계산하기 위함이다. 이

를 통해 계산 속도를 향상시키고 신경망의 학습 효율을 높일 수 있다.

- 활성화 함수로 sigmoid, ReLU, leaky_ReLU, 등을 사용하였는데 최종적으로는 층이 길고 많은 뉴런을 사용하는 레이어를 사용했기 때문에 가장 가벼운 ReLU를 사용하게 되었다.

7.3 Loss Function 지정

```
## Loss Function
def MSE(self, y_pred, y_true):
    # 평균 제곱근 오차(Root Mean Squared Error) 손실 함수
    return ((y_pred - y_true) ** 2).mean()
```

- forward의 출력 딕셔너리의 마지막 값을 y_pred와 실제 값인 y_true를 사용하여 손실값을 계산한다.
- 회귀이므로 MSE로 Loss Function을 지정해주었다.

7.4 역전파 (backward pass)

```
def backward(self, X, y_true):
    # 신경망을 통한 역방향 전파하여 그래디언트 계산
    activations = self.forward(X) # 순방향 전파를 통해 활성화
    # 값 얻기
    gradients = {} # 그래디언트를 저장할 딕셔너리
    m = y_true.shape[0] # 샘플의 개수

    # 출력층의 그래디언트 계산
    dZ = 2 * (activations['A' + str(len(self.params)//2)] -
y_true) / m
    gradients['dW' + str(len(self.params)//2)] =
np.dot(activations['A' + str(len(self.params)//2 - 1)].T, dZ)
    gradients['db' + str(len(self.params)//2)] = np.sum(dZ,
axis=0, keepdims=True) /m
```

```

# 은닉층을 통한 역방향 전파
for i in range(len(self.params)//2 - 1, 0, -1):
    dA = np.dot(dZ, self.params['W' + str(i+1)].T)
    dZ = dA * self.ReLU_derivative(activations['Z' +
str(i)])
    gradients['dW' + str(i)] = np.dot(activations['A' +
str(i-1)].T, dZ)
    gradients['db' + str(i)] = np.sum(dZ, axis=0,
keepdims=True) /m

# 확률적 경사 하강법을 사용하여 매개변수 업데이트
self.sgd(gradients)

```

7.4.1 최종 출력에서의 Weight의 Gradient

7.4.1.1 변수 설명

- L : 손실함수(Loss Function)
- A : 이전 층의 활성화 출력
- W : 가중치 행렬
- Z : 현재 층의 선형 변환 결과
- m : 샘플의 개수

7.4.1.2 수학적 표현

- 마지막 출력에서의 손실함수를 마지막 층에서의 가중치에 대하여 미분하면 다음과 같다.
- 선형 변환 $Z = A \cdot W + b$ 을 사용해서 손실함수의 가중치에 대한 미분값 $dW = \frac{\partial L}{\partial W}$ 을 구한다

- $$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

- 이를 구하기 위해서는 손실함수를 최종 선형 변환에 대해 미분한 값인 $\frac{\partial L}{\partial Z}$ 와 선형함수를 가중치에 대해 미분한 값 $\frac{\partial Z}{\partial W}$ 를 구해야 한다.

- $\frac{\partial Z}{\partial W}$ 는 $Z = A \cdot W + b$ 를 W 에 대해 미분하면 $\frac{\partial Z}{\partial W} = A$ 이고 행렬의 차원을 맞추기 위해 A 를 전치시켜서 $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$ 에 대입하면 아래와 같다.

- $$\frac{\partial L}{\partial W} = A^T \cdot \frac{\partial L}{\partial Z}$$

7.4.1.3 코드 구현

출력값의 손실함수에 대한 Gradient dZ 구하기

- 먼저 $dZ = \frac{\partial L}{\partial Z}$ 를 구하기 위해 최종 출력값에 Loss Function(MSE)를 대입한 값을 미분한다.

```
dZ = 2 * (activations['A' + str(len(self.params)//2)] - y_true) / m
```

- 마지막 출력값을 찾기 위해서 forward에서 저장한 활성화 출력 값의 마지막 값을 찾아야 한다.
- `activations['A' + str(len(self.params)//2)]`
- `self.params` 은 Weights, biases 두 개의 값을 저장했기 때문에 2로 나눈 몫을 찾아서 마지막 활성화 출력값인 예측값을 MSE를 미분한 함수에 넣어서 dZ 값을 구해준다. (m은 샘플의 개수를 의미)

- $$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

A^T 와 dZ 를 dot product하여 Weights에 대한 Gradient 구하기

```
gradients['dW' + str(len(self.params)//2)] = np.dot(activations['A' + str(len(self.params)//2 - 1)].T, dZ)
```

- $$\frac{\partial L}{\partial W} = A^T \cdot \frac{\partial L}{\partial Z}$$

- 마지막 층의 활성화 출력이 저장된 activations 딕셔너리값을 가져와서 Transpose 해준 후 dZ 값과 dot product를 해주면 마지막 층에서의 Gradient를 구할 수 있게 된다.

7.4.2 최종 출력에서의 biases의 Gradient

선형회귀 문제를 해결하는 것을 목표로 하고 있으므로 출력층에서는 Activation Function을 사용하지 않으므로 backward함수에서 마지막 출력과 그렇지 않은 경우를 나누어서 계산하는 것을 확인할 수 있다.

7.4.2.1 변수 설명

- L : 손실함수(Loss Function)
- A : 이전 층의 활성화 출력
- dA : L 에 대한 A 의 Gradient
- W : 가중치 행렬
- Z : 현재 층의 선형 변환 결과
- dZ : L 에 대한 Z 의 Gradient
- m : 샘플의 개수

7.4.2.2 수학적 표현

- 최종 출력에서의 Biases에 대한 Gradient $db = \frac{\partial L}{\partial b}$ 를 구하기 위해 Linear transform을 b 에 대하여 미분해준다.
- 선형 변환 $Z = A \cdot W + b$ 를 b 에 대하여 미분하면 $\frac{\partial Z}{\partial b} = 1$ 이고
- Chain rule을 이용하여

- $$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial b}$$

- $$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial Z} \cdot 1$$

- $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial Z}$, 즉 편향의 그래디언트는 손실 함수에 대한 선형 변환 결과의 그래디언트와 동일하다

- 하지만 편향은 해당 층에서의 Gradient를 구하고 그것의 평균을 사용한다. 그 이유는 데이터의 일반적인 경향을 더 잘 반영하기 위함으로 각각의 Gradient를 다르다면 학습이 불균형적으로 진행되고 과적합의 가능성이 존재하기 때문

7.4.2.3 코드 구현

```
gradients['db' + str(len(self.params)//2)] = np.sum(dZ, axis=0,
keepdims=True) / m
```

- gradients의 가중치 부분을 저장하는 코드이다.
- dz값들을 전부 더하고 keepdims=True를 사용해서 차원을 그대로 유지해준 후 평균을 구하기 위해 샘플의 개수(m)만큼 나누어 준다.

7.4.3 은닉층에서의 Weights, biases의 Gradient 구하기

은닉층에서도 Weights, biases의 Gradient를 구하는 방식은 같지만 Activation Function의 미분을 추가해주어야 한다.

7.4.3.1 변수 설명

- L : 손실함수(Loss Function)
- $A^{[i]}$: 층 i 의 활성화 출력
- $W^{[i]}$: 층 i 의 가중치 행렬
- $Z^{[i]}$: 층 i 의 선형 변환 결과
- $dZ^{[i]}$: 손실함수 L 에 대한 $Z^{[i]}$ 의 Gradient
- $dA^{[i]}$: 손실함수 L 에 대한 $A^{[i]}$ 의 Gradient

7.4.3.2 수학적 표현

- 임의의 층에서 L 의 $W^{[i+1]}$ 와 $b^{[i+1]}$ 에 대한 Gradient를 우리는 구해야 한다.

- $$dW^{[i+1]} = \frac{\partial L}{\partial W^{[i+1]}}$$

- 는 Chain Rule를 사용하여 아래와 같이 표현할 수 있고

- $$dW^{[i+1]} = \frac{\partial L}{\partial Z^{[i+1]}} \cdot \frac{\partial Z^{[i+1]}}{\partial W^{[i+1]}}$$

- $$dW^{[i+1]} = dZ^{[i+1]} \cdot \frac{\partial Z^{[i+1]}}{\partial W^{[i+1]}}$$

- 그러므로 우리는 $dZ^{[i+1]}$ 와 $\frac{\partial Z^{[i+1]}}{\partial W^{[i+1]}}$ 를 구할 필요가 있다.
- 또한 $dZ^{[i+1]}$ 또한 연쇄법칙을 사용하여 아래와 같이 표현할 수 있다.

- $$dZ^{[i+1]} = \frac{\partial L}{\partial Z^{[i+1]}} = \frac{\partial L}{\partial A^{[i+1]}} \cdot \frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}} = dA^{[i+1]} \cdot \frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}}$$

- $$dZ^{[i+1]} = dA^{[i+1]} \frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}}$$

- 그리고 이 식에서 $A^{[i+1]}$ 은 $Z^{[i+1]}$ 가 입력인 활성화 함수의 결과값이므로 $\frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}}$ 부분은 ReLU의 미분 함수를 의미하게 된다.

- $$\frac{\partial Z^{[i+1]}}{\partial A^{[i+1]}} = \begin{cases} 1 & \text{if } Z^{[i+1]} > 0 \\ 0 & \text{if } Z^{[i+1]} \leq 0 \end{cases}$$

- $$dW^{[i+1]} = dZ^{[i+1]} \cdot A^{[i]T}$$

-

- d 를 구하기 위해서는 가 필요한데 를 구하기 위해 다시 Chain rule을 사용하면 아래와 같다.

-

-

- 또한 $Z^{[i+1]} = A^{[i]} \cdot W^{[i+1]} + b^{[i+1]}$ 을 $A^{[i]}$ 에 관해 미분하면 아래의 식을 얻을 수 있다.

- $\frac{\partial Z^{[i+1]}}{\partial A^{[i]}} = W^{[i+1]}$

- $\frac{\partial L}{\partial Z^{[i+1]}} = dZ^{[i+1]}$ 라고 할 수 있으므로

- 그 다음 연쇄법칙을 사용해서 식을 아래와 같이 만들고 위의 식을 이용하면

$$dA^{[i]} = \frac{\partial L}{\partial Z^{[i+1]}} \cdot \frac{\partial Z^{[i+1]}}{\partial A^{[i]}}$$

$$dA^{[i]} = dZ^{[i+1]} \cdot W^{[i+1]T}$$

7.4.3.3 코드 확인

```
# 은닉층을 통한 역방향 전파
for i in range(len(self.params)//2 - 1, 0, -1):
    dA = np.dot(dZ, self.params['W' + str(i+1)].T)
    # dZ = dA * self.leaky_ReLU_derivative(activations['Z' +
+ str(i)])
    dZ = dA * self.ReLU_derivative(activations['Z' +
str(i)])
    gradients['dW' + str(i)] = np.dot(activations['A' +
str(i-1)].T, dZ)
    gradients['db' + str(i)] = np.sum(dZ, axis=0,
keepdims=True) / m
```

- `for i in range(len(self.params)//2 - 1, 0, -1):` 출력층에서부터 첫 번째 은닉층까지 거꾸로 전파되어야하기 때문에 -1을 사용하여 반대로 진행하는 것을 확인할 수 있다.
- 이미 출력층에서의 Gradient는 구했기 때문에 `len(self.params)//2 - 1` 을 사용하여 층을 하나 줄였다.
- $dA^{[i]} = dZ^{[i+1]} \cdot W^{[i+1]T}$ 의 수식 대로 코드는 `dA = np.dot(dZ, self.params['W' + str(i+1)].T)` 을 나타낸다
- $dZ^{[i+1]} = dA^{[i+1]} \frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}}$ 수식의 코드는 `dA * self.ReLU_derivative(activations['Z' + str(i)])` 으로 $A^{[i+1]}$ 은 $Z^{[i+1]}$ 의 활성화 출력이므로 $Z^{[i+1]}$ 에 대해 미분해주면 사용하는 활성화 함수의 미분을 의미하게 된다.

- 그러므로 $\frac{\partial A^{[i+1]}}{\partial Z^{[i+1]}}$ 가 코드 `self.ReLU_derivative(activations['Z' + str(i)])` 를 의미한다고 할 수 있다.
- 이후 은닉층에서의 Gradient는

$$\frac{\partial L}{\partial W^{[i]}} = A^{[i-1]T} \cdot \frac{\partial L}{\partial Z^{[i+1]}} = A^{[i-1]T} \cdot dZ^{[i+1]}$$

이므로 저장된 `activations` 딕셔너리와 연쇄법칙으로 구한 $dZ^{[i+1]}$ 을 통해 계산된다.

- `biases` 또한 $dZ^{[i+1]}$ 값의 평균값을 저장하게 된다.

7.5 Optimizor 지정

```
def sgd(self, gradients, max_grad=5):
    # 확률적 경사 하강법을 사용하여 매개변수 업데이트
    for i in range(1, len(self.params)//2 + 1):
        # 그래디언트 클리핑
        gradients['dW' + str(i)] = np.clip(gradients['dW' + str(i)], -max_grad, max_grad)
        gradients['db' + str(i)] = np.clip(gradients['db' + str(i)], -max_grad, max_grad)
        # 매개변수 업데이트
        self.params['W' + str(i)] -= self.learning_rate * gradients['dW' + str(i)]
        self.params['b' + str(i)] -= self.learning_rate * gradients['db' + str(i)]
```

7.6 활성화 함수 (Activation Function)

활성화 함수

```
def sigmoid(self, z):
    # z 값이 100보다 크면 오버플로를 방지하기 위해 1을 반환
```

```

        z = np.clip(z, -100, 100)
        return 1 / (1 + np.exp(-z))

    def sigmoid_derivative(self, z):
        # 시그모이드 함수의 도함수
        sig = self.sigmoid(z)
        return sig * (1 - sig)

    def ReLU(self, Z):
        # Rectified linear unit 활성화 함수
        return np.maximum(0, Z)

    def ReLU_derivative(self, Z):
        # ReLU 활성화 함수의 도함수
        return (Z > 0).astype(float)

    def leaky_ReLU(self, Z, alpha=0.01):
        # Leaky ReLU 활성화 함수
        return np.maximum(alpha * Z, Z)

    def leaky_ReLU_derivative(self, Z, alpha=0.01):
        # Leaky ReLU 활성화 함수의 도함수
        return (Z > 0) + alpha * (Z <= 0)

```

- 성능을 높이기 위해 여러 활성화 함수들을 사용해보았다.

8. 모델 훈련

```

import numpy as np

# 앞서 정의한 SimpleNN 클래스를 사용하여 인공 신경망 인스턴스 생성
# 입력층 8개, 은닉층 4개 (각각 16, 16, 8, 8개 뉴런), 출력층 1개
nn = SimpleNN(layer_sizes=[8, 64, 64, 32, 16, 8, 4, 1],
               learning_rate=0.0001, initialization='he')

```

```
# 학습을 위한 설정
# 에포크: 전체 데이터 세트에 대한 학습 횟수
epochs = 1000
# patience: 성능 향상이 없을 때 기다리는 에포크 수
patience = 30
# 미니배치 크기: 각 학습 단계에서 사용되는 데이터 샘플 수
batch_size = 64

# Early stopping을 위한 변수 초기화
# 최고의 손실 값을 저장
best_loss = np.inf
# 성능이 향상되지 않는 에포크 수를 세기 위한 변수
no_improvement_count = 0
# 가장 좋은 가중치를 저장하기 위한 변수
best_weights = None

# 검증 데이터의 손실을 모니터링하기 위한 변수 초기화
best_val_loss = np.inf

# 학습 시작
for epoch in range(epochs):
    # 데이터 세트를 미니배치로 나누기
    num_batches = int(np.ceil(X_train.shape[0] / batch_size))
    indices = np.arange(X_train.shape[0])
    np.random.shuffle(indices) # 데이터 세트를 무작위로 섞음

    # 각 미니배치에 대한 학습
    for batch in range(num_batches):
        start = batch * batch_size
        end = min(start + batch_size, X_train.shape[0])
        X_batch = X_train[indices[start:end]]
        y_batch = y_train[indices[start:end]]
```

```
# 순방향 계산
activations = nn.forward(X_batch)

# 손실 계산 (미니배치에 대해서)
loss_train = nn.MSE(activations['A' +
str(len(nn.params)//2)], y_batch)

# 역방향 계산 및 파라미터 업데이트
nn.backward(X_batch, y_batch)

# 검증 데이터에 대한 순방향 계산
activations_val = nn.forward(X_val)

# 검증 데이터에 대한 손실 계산
loss_val = nn.MSE(activations_val['A' +
str(len(nn.params)//2)], y_val)

# Early stopping 검사
if loss_val < best_val_loss:
    best_val_loss = loss_val
    best_weights = nn.params.copy()
    no_improvement_count = 0
else:
    no_improvement_count += 1
    if no_improvement_count >= patience:
        print(f"조기 종료 - 에포크 {epoch}에서 학습 중단!")
        nn.params = best_weights # 최고의 가중치로 복원
        break

# 일정 에포크마다 학습 상황 출력
```

```
    if epoch % 50 == 0:
        print(f'에포크 {epoch}, 훈련 손실: {loss_train}, 검증 손실: {loss_val}')
```

```
# 학습 완료 후 테스트 데이터에 대한 예측 수행
y_pred = nn.forward(X_test)['A' + str(len(nn.params)//2)]
```

```
# 결과 출력
```

```
# print('학습된 가중치 및 편향:', nn.params)
```

```
print('예측값:', y_pred)
```